

CityGrocer Web Application

Complete End-to-End Architectural Blueprint, Component Workflow & Development Roadmap

PROJECT METADATA & COMPONENT SPECIFICATION

- Target Platforms: Mobile Phone (iOS/Android), Tablet & Laptop
 - Tech Stack: React 18, TypeScript, Tailwind CSS, Express / Flask REST API
- Currency: Indian Rupee (¹ INR)
 - Architecture: In-Memory Multi-Store Matrix & Heuristic Optimizer

1. Project Vision & Core Objectives

High-level goals and problem statement addressed

CityGrocer is a responsive hyper-local grocery intelligence platform designed to eliminate overpaying for household necessities. It tracks and indexes daily prices across 6 major competing grocery supermarket chains in real time, empowering consumers and small businesses to compare unit prices, find exact store aisles, and optimize multi-stop shopping carts for maximum savings.

Core Value Propositions & Differentiators

- Cross-Store Price Transparency: Real-time price disparity tracking across 40 staple commodities and 223+ verified in-store price points.
- Multi-Store Trip Optimizer: Combinatorial cart splitter computing maximum cash savings against travel effort for single vs multi-store trips.
- Hyper-Local Zone Filter: City neighborhood targeting across Central, T. Nagar, Anna Nagar, Adyar, Velachery & Porur.
- Direct Item Hunting & Aisle Locator: Pinpoints exact physical aisle numbers (e.g. Aisle 3 - Dairy, Aisle 2 - Grains).
- Crowdsourced Verification: User price submission pipeline with confidence scoring and timestamp badges.

2. System Architecture & Tech Stack Checklist

Layered component structure and dependencies

Frontend Client Tier (Single Page Application & Mobile Viewport)

- Framework & Language: React 18 with strict TypeScript type models.
- Build Tool: Vite with tree-shaking and production bundling.
- Design System: Tailwind CSS with responsive mobile-first grid and custom color palettes.
- Iconography: Lucide-React vector icons for all actions and badges.
- Interactive Visuals: Custom SVG Cartographer map with pinpoint store radar & coordinate projection.

Backend API & Serving Tier (Express / Flask REST API)

- Runtime: Node.js 22 LTS with Express.js REST API middleware & CORS support.
- Core Endpoints: /api/stores, /api/products, /api/compare/:id, /api/cheapest/:id, /api/report-price.
- File Distribution: Direct static bundle download (/citygrocer-complete-project.zip) and PDF download (/citygrocer-roadmap.pdf).

3. Required Frontend Components & Roles

Full breakdown of all 11 core UI components

- **Header (Header.tsx):** Top branding, zone selector dropdown, live currency badge (¹ INR), global button, and desktop cart counter badge.
- **PriceComparisonView (PriceComparisonView.tsx):** Primary dashboard featuring category filters (Produce, Dairy, Grains, Oils, Spices), search matrix (Price Low->High, Highest Savings, Most Verified), and floating quick-trip action bar.
- **ProductCard (ProductCard.tsx):** Compact item cards rendering store badges, lowest vs highest price delta, aisle location tag, stock availability calculations, and 1-click cart addition.
- **ProductDetailModal (ProductDetailModal.tsx):** Deep-dive drawer displaying store-by-store comparison tables, price history trends, aisle navigation timestamps.
- **BasketOptimizerModal (BasketOptimizerModal.tsx):** Combinatorial trip calculation engine. Compares: Option A (Single Cheapest Store) vs C item-by-item shopping checklists and net ¹ savings.
- **CityMapView (CityMapView.tsx):** Interactive SVG vector map plotting all 6 retail locations with live distance rings, store details, inventory preview approximations.

- **DealsRadarView (DealsRadarView.tsx):** Analytics engine highlighting items with extreme price variance.
- **ItemHunterView (ItemHunterView.tsx):** Direct buyer catalog enabling filtering by store chain, item classification, and instant buy routing.
- **PriceReportModal (PriceReportModal.tsx):** Community submission form for updating in-store shelf prices, attaching store tags, and submitting ZIP/PDF downloads.
- **ShareAppModal (ShareAppModal.tsx):** 1-click link copying, WhatsApp/Telegram/Email sharing, instant QR code generator for phone cameras.
- **ApiExplorerView (ApiExplorerView.tsx):** Developer documentation workbench with live cURL examples, schema tables, and interactive endpoints.

4. Data Schemas & Dataset Engineering

Structured TypeScript models and relationships

Product & StorePrice Schema (/src/types/grocery.ts)

- Product { id, name, tamilName, category, unit, marketAvgPrice, prices: Record<StoreId, StorePrice> }
- StorePrice { currentPrice: number, stockStatus: "IN_STOCK" | "LOW_STOCK" | "OUT_OF_STOCK", aisleLocation: string, verifiedDate: string, discountPercent?: number }
- Store { id: string, name: string, type: "Supermarket" | "Hypermarket" | "Wholesale", address: string, zoneId: string, coordinates: { lat, lng }, rating: number, priceTier: "Budget" | "Moderate" | "Premium" }
- CityZone { id: string, name: string, pincode: string, coordinates: { lat, lng } }

5. Chronological 7-Phase Development Roadmap

Step-by-step roadmap from inception to deployment

Phase 1: Requirements Gathering & UX Wireframing

- Step 1.1: Define target user personas (Budget shoppers, meal planners, small business owners).
- Step 1.2: Map out user flow: Search Item -> Compare Stores -> Inspect Aisle -> Optimize Basket -> Share/Print.
- Step 1.3: Build wireframes for smartphone (375px), tablet (768px), and desktop (1280px+).

Phase 2: Data Schema Design & Mock Catalog Seeding

- Step 2.1: Establish TypeScript models for products, stores, city zones, and price reports.
- Step 2.2: Seed 40 essential commodities with realistic Indian supermarket prices across 6 chains.
- Step 2.3: Define spatial coordinates and neighborhood zones for hyper-local filtering.

Phase 3: Core Algorithmic Engines Development

- Step 3.1: Build Multi-Store Greedy Optimizer: Calculate total trip cost across N stores vs single-store baseline.
- Step 3.2: Implement Deals Radar Analyzer: Compute price spread variance and markdowns.
- Step 3.3: Build Search & Sorting Index: Tokenize keywords, filter by price/category/zone.

Phase 4: Responsive UI Component Assembly

- Step 4.1: Assemble Header, PriceComparisonView, ProductCard, and ProductDetailModal.
- Step 4.2: Implement touch-optimized mobile navigation dock with bottom safe-area padding.
- Step 4.3: Build the SVG City Cartographer map with interactive store pin overlays.

Phase 5: Backend REST API & Express Server Integration

- Step 5.1: Implement Express / Flask endpoints (/api/stores, /api/products, /api/compare/:id).
- Step 5.2: Connect Price Update submission pipeline with server-side validation.
- Step 5.3: Mount Vite development middleware and production static fallback routes.

Phase 6: Multi-Device Optimization & Accessibility Audit

- Step 6.1: Validate touch targets (min 44px) on mobile iOS and Android browsers.
- Step 6.2: Check color contrast ratios (WCAG AA standard) for emerald, slate, and amber elements.
- Step 6.3: Ensure modals handle background scroll locking and escape-key dismissal smoothly.

Phase 7: Packaging, Public Sharing & Cloud Deployment

- Step 7.1: Generate public sharable URLs with dynamic origin resolution.
- Step 7.2: Build automated ZIP archiver bundling all source code, datasets, and server assets.
- Step 7.3: Generate official PDF Architecture & Workflow roadmap for stakeholders and developer teams.

6. REST API Endpoint Reference

Standard HTTP REST endpoints for client integrations

- **GET /api/stores:** Returns complete list of 6 supermarket chains with ratings, price tiers, and coordinates.
- **GET /api/products:** Accepts ?category=X&zone=Y&search=Z queries. Returns filtered and ranked grocery catalogue.
- **GET /api/compare/:id:** Returns detailed store-by-store price breakdown, savings delta, and aisle information for a given item.
- **GET /api/cheapest/:id:** Returns the single lowest price store and unit rate for rapid 1-click buy lookups.
- **POST /api/report-price:** Payload { productId, storeId, reportedPrice, aisleLocation }. Updates database with new crowd price report.
- **GET /citygrocer-complete-project.zip:** Streams and downloads the full project ZIP bundle containing all files and datasets.
- **GET /citygrocer-roadmap.pdf:** Streams and downloads this formal Architecture & Roadmap PDF document.

CityGrocer Technical Roadmap - Verified Architecture Specification

This PDF contains the full workflow, technical architecture, and component checklist for the CityGrocer cross-device responsive grocery comparison and basket optimization application.